CasePilot V2

AI-Powered Security Investigation Assistant

High Level Design (HLD)

Document Type	**Technical Proposal**
Version	**2.1**
Author	**Bhupender Sharma**
Date	**June 2026**
Status	**Final**
Audience	**Architects, Engineering Leads, Security Teams**

Table of Contents

1. Executive Summary

CasePilot V2 is an AI-powered investigation platform designed to assist security analysts in understanding and investigating fraud, risk, and security-related cases.

The platform integrates with existing enterprise systems, automatically generates investigation summaries, and provides a conversational interface for analysts to ask questions about a case.

The solution follows a service-oriented architecture and centralizes all AI interactions through a dedicated AI Gateway Service, allowing the organization to remain provider-agnostic while maintaining security, observability, and operational control.

2. Problem Statement

Security analysts often need to review information spread across multiple systems:

- Case Management System
- Customer Management System
- Payment Platform
- Session & Device Intelligence Platform
- Risk & Fraud Detection Systems

As a result:

- Investigations are slow
- Context gathering is manual
- Summaries are inconsistent
- Knowledge is difficult to transfer
- Analysts spend more time collecting information than analyzing it

CasePilot aims to reduce investigation time by automatically aggregating information and presenting it through AI-generated summaries and conversational interfaces.

3. Goals

Functional Goals

- Automatic case summarization
- AI-powered chat assistant
- Multi-model LLM support
- Cost tracking and monitoring
- Secure processing of customer data

Non-Functional Goals

- Scalability
- Reliability
- Security
- Auditability
- Provider independence
- Cost optimization

4. High-Level Architecture

The system consists of three primary business capabilities:

- Summary Generation Flow - Generates AI summaries when case events occur
- Chat & Q&A Flow - Allows analysts to interact with case data using natural language
- Cost & Usage Tracking - Tracks AI usage, token consumption, and operational costs

A dedicated AI Gateway Service centralizes all communication with LLM providers.

5. Architecture Principles

Separation of Responsibilities

Each service owns a specific business capability.

Event-Driven Processing

Case updates are processed asynchronously through Kafka.

Provider Agnostic Design

Business services never directly call OpenAI, Claude, or Gemini. All AI interactions occur through AI Gateway.

Security First

PII is removed before information is sent to external AI providers.

Observability

All requests, failures, and token consumption are tracked.

6. Core Services

Service	Responsibility
Summary Service	Generates case summaries
Chat Service	Handles analyst conversations
AI Gateway Service	Manages LLM providers
Cost Service	Tracks usage and costs
Kafka	Event transport
Redis	Session caching
Summary Database	Stores summaries
Chat Database	Stores conversations
Usage Database	Stores AI metrics

7. Summary Generation Service - Deep Dive

The Summary Generation Service is the primary value-delivery component of CasePilot. It consumes case lifecycle events from Kafka, aggregates investigation data from four upstream systems, applies security controls, and produces structured AI-generated investigation summaries.

7.1 Kafka Consumer Design

The service uses a Kafka consumer group with configurable concurrency.

Parameter	Value	Rationale
Consumer Group	casepilot-summary-consumer	Dedicated consumer group
Topic	case-events	All case lifecycle events
Partition Strategy	By caseId	Ordered processing per case
Auto Commit	Disabled	Manual commit after persistence
Max Poll Records	10	Bounded batch size
Concurrency	3	Parallel partition consumers

Why manual commit?

Auto-commit risks marking a message as consumed before processing completes. If the service crashes mid-processing, the message is lost. With manual commit, the offset is committed only after the summary is persisted.

Input Event Schema:

```
{
  "eventId": "evt-a8f3c912",
  "caseId": "CASE-1001",
  "eventType": "CASE_CREATED",
  "timestamp": "2026-06-10T10:00:00Z",
  "source": "case-management-service",
  "version": "1.0"
}
```

Event Validation: Before processing, each event is validated against schema, checked for duplicates using Redis (TTL 24h), and verified against the event type whitelist. Invalid events are routed to the DLQ.

7.2 Data Aggregation Strategy

The service makes parallel HTTP calls to four upstream services. Parallel execution is critical because sequential calls would add 2-4 seconds of latency per additional system.

Service	Timeout	Retry	On Failure
Case Management	3s	1 retry	Abort - case data required
Customer System	2s	1 retry	Proceed with partial data
Payment System	3s	1 retry	Proceed with partial data
Session System	2s	1 retry	Proceed without session data

> **Why is Case Management mandatory?**
> Without case details (alerts, risk score, case status), a summary would be meaningless. Other data sources enrich the summary but are not strictly required.

Aggregated Context Model:

```
{
  "caseId": "CASE-1001",
  "caseStatus": "OPEN",
  "riskScore": 82,
  "alerts": [
    {
      "alertId": "ALR-501",
      "type": "VELOCITY_CHECK",
      "severity": "HIGH",
      "description": "15 transactions in 2 hours from 3 countries"
    }
  ],
  "customer": {
    "customerId": "CUST-2200",
    "accountAge": "P2Y3M",
    "kycStatus": "VERIFIED",
    "riskTier": "MEDIUM"
  },
  "transactions": [ ... ],
  "sessions": [ ... ],
  "dataCompleteness": {
    "caseData": true,
    "customerData": true,
    "paymentData": true,
    "sessionData": true
  }
}
```

The dataCompleteness field tells the LLM which data sources were available. If sessionData is false, the summary includes a note: "Session data was unavailable during summary generation."

7.3 PII Redaction Pipeline

PII redaction occurs in two stages before data reaches the AI provider.

Stage 1: Pattern-Based Redaction (Regex)

Pattern	Example	Replacement
Email	john@example.com	[EMAIL_1]
Phone	+44 7911 123456	[PHONE_1]
IBAN	DE89 3704 0044 0532 0130 00	[IBAN_1]
Credit Card	4111 1111 1111 1111	[CARD_1]
IP Address	192.168.1.100	[IP_1]

Stage 2: Named Entity Recognition (NER via Microsoft Presidio)

Entity	Example	Replacement
Person Name	John Smith	[PERSON_1]
Address	123 Baker Street, London	[ADDRESS_1]
Organization	Acme Corp	[ORG_1]

What is NOT redacted (and why)

Transaction amounts, timestamps, risk scores, alert types, and country codes are preserved. The LLM needs these for risk analysis, temporal pattern detection, and cross-border investigation.

Re-identification: After the LLM generates the summary, placeholders are replaced back with original values using an in-memory mapping. This mapping is never persisted or logged.

7.4 Context Window Management

A fraud case can contain 200+ transactions. Sending all of them would exceed the context window and waste tokens.

Token Budget Allocation

Component	Budget	Notes
System prompt	1,500 tokens	Fixed
Case metadata	1,000 tokens	Fixed
Customer profile	500 tokens	Fixed
Alerts	1,500 tokens	~15 alerts max
Transactions	4,000 tokens	~40 transactions
Sessions	1,500 tokens	~15 sessions
Response buffer	4,000 tokens	For generated summary
Total	14,000 tokens	

Entity Prioritization

When entities exceed their budget, they are ranked by priority score:

```
priority = 0.4 * risk_score + 0.25 * recency + 0.2 * amount_normalized + 0.15 * anomaly_count
```

Top-K entities per type are included. Remaining entities are summarized as a count with totals.

Large Case Strategy (200+ entities)

For exceptionally large cases, a map-reduce approach is used:

- Chunk entities into groups of 40
- Generate a mini-summary for each chunk
- Merge mini-summaries into a final summary

This costs ~3x more tokens but ensures no significant information is lost.

7.5 Prompt Engineering

The prompt is structured, versioned, and stored in a Prompt Repository.

System Prompt (v2.3)

```
You are a fraud investigation assistant for a financial institution.

Your task: analyze the provided case data and produce a structured
investigation summary.

RULES:
1. Base your analysis ONLY on the provided data
2. Never invent transactions, alerts, or entities not in the input
3. If data is missing, explicitly state what is unavailable
4. Use professional language suitable for compliance documentation
5. Include specific transaction IDs, amounts, and timestamps
6. Provide a risk assessment with supporting evidence

OUTPUT FORMAT: JSON with fields:
  overview, keyFindings, riskAssessment, timelineOfEvents,
  entitiesInvolved, recommendedActions, dataGaps
```

> **Why JSON output?**
> Structured JSON allows the frontend to render each section independently (risk assessment as a badge, timeline as a visual). It also enables programmatic validation.

Prompt Version History

Version	Change	Date
v1.0	Initial prompt	2026-05-01
v2.0	Added timeline of events	2026-05-15

v2.1	Added data gaps section	2026-05-20
v2.3	Refined risk assessment format	2026-06-01

7.6 Response Validation (Anti-Hallucination)

LLMs can generate plausible but incorrect information. In fraud investigation, a hallucinated transaction could lead to a wrong decision.

Validation Rules

- Schema validation - response must match expected JSON structure
- Entity cross-reference - every transaction ID in the summary must exist in input data
- Amount verification - stated amounts are cross-checked against source data
- Completeness check - all required sections must be present
- Content safety - no prompt leakage (system prompt in output)

On Validation Failure

Issue	Action
Missing optional field	Auto-fix and proceed
Entity hallucination	Remove reference, add warning
Invalid JSON / empty	Retry (max 2 retries)
Persistent failure	Store "failed" record, notify analyst

7.7 Persistence and Versioning

```
CREATE TABLE case_summary (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    case_id         VARCHAR(100) NOT NULL,
    version         INTEGER NOT NULL DEFAULT 1,
    summary_json    JSONB NOT NULL,
    model_name      VARCHAR(50) NOT NULL,
    model_provider  VARCHAR(50) NOT NULL,
    prompt_version  VARCHAR(20) NOT NULL,
    input_tokens    INTEGER NOT NULL,
    output_tokens   INTEGER NOT NULL,
    cost_usd        DECIMAL(10,6) NOT NULL,
    processing_ms   INTEGER NOT NULL,
    data_completeness JSONB NOT NULL,
    status          VARCHAR(20) NOT NULL DEFAULT 'COMPLETED',
    created_at      TIMESTAMP NOT NULL DEFAULT NOW(),
    CONSTRAINT uq_case_version UNIQUE (case_id, version)
);
```

Why JSONB?

The summary structure evolves as prompt versions change. JSONB allows schema flexibility without migrations. PostgreSQL JSONB supports indexing and querying individual fields.

7.8 Reliability Design

Retry Policy

Attempt	Wait	Action
1	Immediate	Process event
2	2 seconds	Retry
3	4 seconds	Retry
After 3	-	Route to DLQ

Idempotency

Key: caseId + eventId. Before processing, the service checks Redis. After success, the key is stored with 24h TTL. This prevents duplicates from Kafka at-least-once delivery.

7.9 Performance Targets

Metric	Target
End-to-end latency (P50)	< 8 seconds
End-to-end latency (P95)	< 15 seconds
Data aggregation (P95)	< 3 seconds
LLM response (P95)	< 10 seconds
Success rate	> 99%
Throughput	100 summaries/minute

8. AI Gateway Service - Deep Dive

The AI Gateway is the single point of contact between CasePilot business services and external LLM providers. It encapsulates all AI-specific concerns: provider routing, retry logic, circuit breaking, failover, token tracking, cost calculation, and response validation.

No business service communicates directly with an LLM provider.

8.1 API Contract

Generate Summary

```
POST /api/v1/ai/generate
Content-Type: application/json
Authorization: Bearer <service-token>
X-Correlation-Id: corr-abc123
X-Idempotency-Key: CASE-1001:v2.3:evt-a8f3c912

{
  "requestType": "SUMMARY",
  "caseId": "CASE-1001",
  "context": { ... },
  "promptVersion": "v2.3",
  "maxTokens": 4000,
  "temperature": 0.2,
  "responseFormat": "json",
  "priority": "NORMAL",
  "callerService": "summary-service"
}
```

Response

```
{
  "requestId": "req-xyz789",
  "status": "SUCCESS",
  "provider": "azure-openai",
  "model": "gpt-4o",
  "response": { ... },
  "usage": {
    "inputTokens": 3200,
    "outputTokens": 1850,
    "totalTokens": 5050,
    "costUsd": 0.065
  },
  "latencyMs": 4200,
  "promptVersion": "v2.3"
}
```

8.2 Request Processing Pipeline

Every request passes through a 7-stage pipeline:

Stage 1: Request Validation

Validates required fields, maxTokens range (100-8000), temperature (0.0-1.0), payload size (< 500KB), and caller service registration.

Stage 2: Rate Limit Check

Per-service rate limits using Redis. Summary Service: 200 req/min. Chat Service: 500 req/min. Exceeded limits return HTTP 429 with Retry-After header.

Stage 3: Provider Selection

Routes requests based on type, cost, and availability. Summary requests use GPT-4o-mini (10x cheaper). Chat requests use GPT-4o (stronger reasoning). Long-context requests route to Claude.

Stage 4: Prompt Assembly

Loads prompt template, injects context data, counts tokens. If prompt exceeds 80% of context window, the Context Builder reduces the context.

Stage 5: LLM Execution

Executes with retry and failover logic (detailed below).

Stage 6: Response Validation

Verifies JSON format, completeness, and safety. Alerts on 50%+ token deviation.

Stage 7: Usage Recording

Fires usage event to Kafka for cost tracking (asynchronous, fire-and-forget).

8.3 Provider Routing Policy

Request Type	Model	Provider	Rationale
Summary	GPT-4o-mini	Azure OpenAI	Structured extraction; 10x cheaper
Chat	GPT-4o	Azure OpenAI	Complex reasoning needed
Chat (>100K tokens)	Claude 3.5 Sonnet	Anthropic	200K context window
Fallback	Gemini 1.5 Pro	Google	Third-tier fallback

8.4 Circuit Breaker Design

Each provider has an independent circuit breaker:

Parameter	Value
Failure threshold (to open)	5 consecutive failures
Success threshold (to close)	3 successes from half-open
Open state duration	30 seconds

Circuit Breaker States

```
CLOSED (normal)
    |
    5 consecutive failures
    |
    v
OPEN (fail fast, skip provider)
    |
    30 seconds
    |
    v
HALF-OPEN (allow 1 test request)
    |
    +-- Success --> CLOSED
    +-- Failure --> OPEN
```

When a circuit is OPEN, the Provider Router automatically skips that provider and routes to the next in the failover chain. The caller is unaware of the failover.

8.5 Retry and Failover Strategy

Error	Action
HTTP 429 (Rate Limit)	Respect Retry-After, retry same provider

HTTP 500/502/503	Retry once, then failover
Timeout (>30s)	Immediate failover to next provider
HTTP 401/403	Do NOT retry. Alert operations.
Network error	Failover to next provider
Circuit OPEN	Skip provider, use next in chain

Failover Chain

```
Azure OpenAI (primary)
      |
      v  (429 / 5xx / timeout)
Anthropic Claude (secondary)
      |
      v  (failure)
Google Gemini (tertiary)
      |
      v  (failure)
Return error to caller
```

8.6 Cost Calculation Engine

Model	Input / 1M tokens	Output / 1M tokens
GPT-4o-mini	$0.15	$0.60
GPT-4o	$2.50	$10.00
Claude 3.5 Sonnet	$3.00	$15.00
Gemini 1.5 Pro	$1.25	$5.00

Budget Controls

Control	Threshold	Action
Per-case cap	$0.50	Switch to cheapest model
Per-analyst daily	50 questions	Soft limit with override
Global daily	$500	Alert engineering team

Projected Monthly Cost (1,000 cases/day)

Operation	Model	Volume/Day	Monthly Cost
Summaries	GPT-4o-mini	1,200	$21
Chat Q&A	GPT-4o	3,000	$930
Total		4,200	$951

8.7 Observability

Prometheus Metrics

Metric	Type	Labels
ai_gateway_requests_total	Counter	provider, model, status
ai_gateway_latency_seconds	Histogram	provider, model
ai_gateway_tokens_total	Counter	provider, direction
ai_gateway_cost_usd_total	Counter	provider, model
ai_gateway_circuit_state	Gauge	provider
ai_gateway_failover_total	Counter	from, to

Alerts

Alert	Condition	Severity
High error rate	> 5% errors in 5 min	P2
Circuit open	Any provider circuit opens	P2
Cost spike	Daily cost > 2x average	P3
All providers down	All circuits open	P1

9. Chat Service

The Chat Service allows analysts to ask questions about a case using natural language. It leverages previously generated summaries and conversation history.

Request Flow

- Receive analyst request
- Load conversation history from Redis (last 10 turns)
- Load latest case summary from Summary DB
- Build conversational context
- Apply PII redaction
- Call AI Gateway
- Store response in Chat DB
- Stream response to analyst via SSE

Redis Session Cache

```
Key:   chat:{caseId}:{userId}
TTL:   24 Hours
Value: Last 10 conversation turns
```

10. Security Design

Control	Implementation
Authentication	Microsoft Entra ID
Authorization	Role-Based Access Control (RBAC)
PII Protection	Two-stage redaction (regex + NER)
Audit Logging	All AI requests logged with correlation ID
Secrets	Azure Key Vault

11. Monitoring & Observability

Service	Key Metrics
Summary Service	Processing latency, success rate, DLQ depth
AI Gateway	Provider usage, token consumption, cost/request, circuit state
Chat Service	Active sessions, response times, user activity

12. Future Enhancements

- Retrieval Augmented Generation (RAG) with vector search
- Prompt versioning UI with A/B testing
- Human feedback loop for summary quality
- Multi-region deployment
- Streaming responses for summary generation

13. Conclusion

CasePilot V2 provides a scalable and secure AI-assisted investigation platform.

The Summary Generation Service automates investigation summaries by aggregating information across enterprise systems, applying PII redaction, managing context windows, and validating LLM outputs against hallucination.

The AI Gateway Service centralizes all AI interactions, providing provider independence through a 7-stage request pipeline, circuit breaker failover, cost tracking, and comprehensive observability.

Together, these services establish a foundation for enterprise-grade AI adoption within security and fraud investigation workflows.